🧩 1. ¿Qué es una dirección IP?

Completar tabla:

Concepto	Definición simple	Nivel técnico breve	Ejemplo
IP	Es el "nombre" único que tiene cada dispositivo en una red. Sin él, los datos no saben a dónde ir	Protocolo de comunicación que asigna una dirección numérica a cada dispositivo.	Tu computador en casa tiene una IP como `192.168.1.5`.
IPv4	La versión original de las direcciones IP. Usa cuatro números separados por puntos, cada uno entre 0 y 255.	Formato de 32 bits que puede acomodar algo más de 4 mil millones de direcciones únicas. AWS	es como si una ciudad creciera tanto que se agotaran todos los números de casa posibles.
IPv6	Simplemente inventaron un sistema nuevo con combinaciones casi infinitas.	Formato de 128 bits que puede acomodar más de 10^{38} direcciones únicas.	2001:0db8:0000:0000:0000:0000:0000:0053

⚙️ 2. Tipos de IP (clave para developers)

Como developer: tu servidor en producción necesita IP estática — si cambiara, nadie podría encontrar tu API. Tu conexión de casa es dinámica porque no importa que cambie.

IP Pública vs Privada

	Pública	Privada
Qué es	La que ve internet	La que ve tu red local
Quién la asigna	Tu proveedor de internet	Tu router
Ejemplo	`99.48.227.227`	`192.168.1.5`
Analogía	Dirección de tu edificio	Número de tu departamento

Tu casa tiene **una sola IP pública** pero todos los dispositivos adentro tienen su propia IP privada. El router hace la traducción.

IP Estática vs Dinámica

	Estática	Dinámica
Qué es	Siempre la misma IP	Cambia cada cierto tiempo
Quién la usa	Servidores, APIs	Usuarios normales, casas
Ejemplo	`api.miapp.com` → siempre `99.48.10.1`	Tu casa hoy `200.10.5.3`, mañana `200.10.5.8`

Como developer: tu servidor en producción necesita IP estática — si cambiara, nadie podría encontrar tu API. Tu conexión de casa es dinámica porque no importa que cambie.

Tipo	¿Qué es?	¿Cuándo se usa?	Ejemplo real
Pública	La IP que ve internet. Es única en el mundo.	Cuando tu dispositivo se comunica fuera de tu red.	Tu router en casa tiene `200.10.5.3` — esa es la que ve Google cuando navegas.
Privada	La IP interna que asigna tu router. No es visible desde internet.	Dentro de una red local: casa, oficina, data center.	Tu computador tiene `192.168.1.5`, tu celular `192.168.1.6` — solo tu router los ve.
Estática	IP fija que nunca cambia. Hay que configurarla manualmente.	Servidores, APIs, bases de datos en producción.	`api.miapp.com` siempre apunta a `99.48.10.1` — si cambiara, la API desaparece.
Dinámica	IP que cambia cada vez que te conectas. La asigna automáticamente el router o el ISP.	Usuarios normales, conexiones de casa o móvil.	Hoy tu casa tiene `200.10.5.3`, mañana puede ser `200.10.5.81` — no importa.

💻 3. Conexión directa con desarrollo (CRÍTICO)

1.-

En Local (localhost)

Cuando trabajas en tu propia computadora, el backend usa la **IP de Loopback**: `127.0.0.1`.

- **Propósito:** Es una dirección especial que le dice al sistema operativo: "No busques en internet, envíame los datos a mí mismo".
- **Privacidad:** Solo es accesible desde el hardware físico donde se está ejecutando.

En Producción (Servidor en Internet)

Cuando subes tu app a un servicio (como AWS, Google Cloud o un VPS), el backend utiliza una **IP Pública**.

- **Propósito:** Es una dirección única en todo el mundo (ejemplo: `157.240.22.35`). Permite que cualquier dispositivo con internet encuentre tu servidor.
- **DNS:** Normalmente, no escribimos la IP, sino un dominio (como `mi-app.com`) que apunta a esa IP pública.

2.-

El término **localhost** es un nombre de dominio que siempre resuelve a "yo mismo".

- Si tú escribes `localhost:3000` en tu PC, tu navegador busca el puerto 3000 en **tu** sistema.
- Si tu amigo escribe `localhost:3000` en su PC, su navegador busca el puerto 3000 en **su** sistema.

Para que alguien más vea tu proyecto local, tendrías que usar tu **IP Privada** (la que te asigna tu router, ej: `192.168.1.15`) y ambos deben estar en la misma red Wi-Fi, o usar herramientas de túnel como **ngrok**.

3.-

Este comando es el motor de arranque de la mayoría de los proyectos modernos (Node.js, React, Vite). Esto es lo que sucede detrás de escena:

- **Lectura del Script:** El sistema busca en tu archivo `package.json` qué comando real debe ejecutar (normalmente algo como `nodemon server.js` o `vite`).
- **Asignación de Puerto:** El proceso solicita al sistema operativo permiso para "escuchar" en un puerto específico (por defecto suele ser el 3000, 5173 o 8080).
- **Levantamiento del Servidor:** El software se queda "despierto" esperando peticiones HTTP. Mientras la terminal esté abierta, el servidor está vivo.
- **Vinculación a la IP:** Por defecto, se vincula a `127.0.0.1`. Si intentas entrar y te da error, puede ser porque el puerto ya está ocupado por otro programa o porque el script tiene un error de sintaxis en el código.

4.-¿Qué IP usa una base de datos en la nube?

IP Privada: Es la que usa el 99% de las veces. Vive dentro de una red cerrada (**VPC**) y solo tu backend puede verla. Es invisible para el resto de internet.

🌍 4. Caso práctico real (debugging)

Escenario:

> “Tu frontend funciona, pero no logra conectarse al backend.”

Responder:

> ¿Qué revisarías primero?

¿Podría ser problema de IP? ¿por qué?

1.- **Sí, puede ser un problema de IP**. El Frontend siempre necesita la dirección (IP o dominio) del Backend para enviarle datos; si la IP es errónea, la conexión falla.

- **En Local:** El Frontend debe apuntar a `127.0.0.1:puerto` (tu PC).
- **En Producción:** El Frontend debe apuntar a la **IP Pública** o dominio del servidor.
- **CORS:** A veces la IP es correcta, pero el Backend bloquea al Frontend por seguridad.

¿Qué revisarías primero?

Mi respuesta:la ip donde esta apuntando la IP, o si hay error en el bakend con la conexion o la sintaxis

Revisaría primero la **URL/IP en el Frontend** para asegurar que apunta al destino correcto (local o nube). Luego, verificaría los **Logs del Backend** en busca de errores de sintaxis o de servidor caído. Es un error común: si el backend no corre, la IP no responderá aunque sea la correcta. Ambas capas deben estar sincronizadas en el mismo **Puerto** para que exista conexión. Tu lógica de "revisar la conexión" es el camino profesional estándar.

¿Qué errores típicos ocurren? (ej: IP mal configurada, puerto, firewall)

IP Incorrecta: El Frontend apunta a `localhost` en producción o a una IP de la nube en local.

Puerto Diferente: El Backend corre en el `4000` pero el Frontend pide datos al `3000`.

Firewall/Nube: El proveedor de la nube tiene el puerto cerrado y bloquea las peticiones externas.

CORS: El navegador bloquea la conexión porque el Backend no tiene permiso para aceptar al Frontend.

Servidor Apagado: El proceso de Node/Python se cayó y la IP no tiene nada que responder.

🧪 5. DHCP y DNS (lo mínimo que TODO developer debe saber)

DHCP: Asigna automáticamente una **IP dinámica** a tu dispositivo al conectarte a una red (te da "tu dirección" temporal).

DNS: Traduce nombres de dominio (google.com) en direcciones IP numéricas que las máquinas entienden (es el "directorio telefónico").

Flujo Google.com:

1. El navegador pregunta al **DNS**: "¿Cuál es la IP de google.com?".
2. El DNS responde con la **IP Pública** (ej: `142.250.190.46`).
3. El navegador usa esa IP para pedir los archivos al **servidor de Google**.
4. El servidor envía el HTML/JS y ves la página.

6. Analogía obligatoria

IP = Dirección postal: Es el número exacto (Calle Falsa 123) que identifica tu casa en el mapa del mundo para que te lleguen los paquetes.

DNS = Contactos del celular: Es el nombre (ej: "Mamá") que usas para no tener que memorizar su número de teléfono real.

DHCP = Recepcionista de hotel: Es quien te entrega una llave y te asigna el número de habitación (IP) apenas llegas, ya que no eres dueño de la habitación para siempre.